

ERA-Engine (Godot) 介绍

ERA-Engine Project

ERA-Engine Project 是一个最初计划用于 [ERA 游戏](#) 的现代游戏引擎的开源项目。我们希望能够创造使开发者、口上和文本作者、非专业玩家都能够轻松开发 ERA 游戏的引擎，并且支持现代程序语言和功能。

ERA-Engine (Godot)

ERA-Engine (Godot) 是以 [Godot 游戏引擎](#) 为基础的版本，也是我们当前正在主要开发的版本。

支持的语言

- [C#](#)
- [GDScript](#)

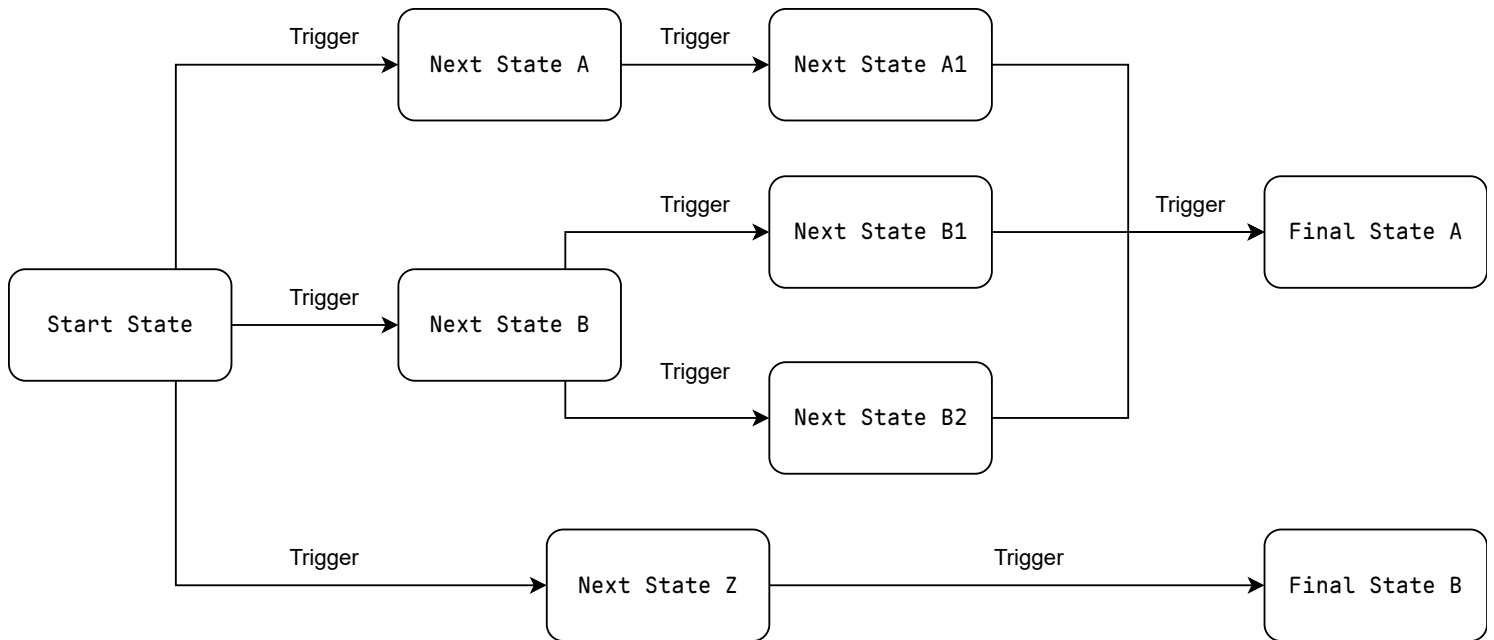
ERA-Engine (Godot) 的设计

设计原则

有限状态机 (FSM)

有限状态机 (Finite State Machine, FSM) 是一个由有限数量的状态组成的系统，**通过在不同状态之间转换来运行**。这是一个用于设计计算机程序和顺序逻辑过程的计算模型。

在这样的模型 (或者说程序) 中，所有的数据、状态和其他信息都被称之为**状态 (state)**，在任何一个给定的时刻，系统总是只有有限个可能的状态——这也是这一架构被称为有限状态机的原因。

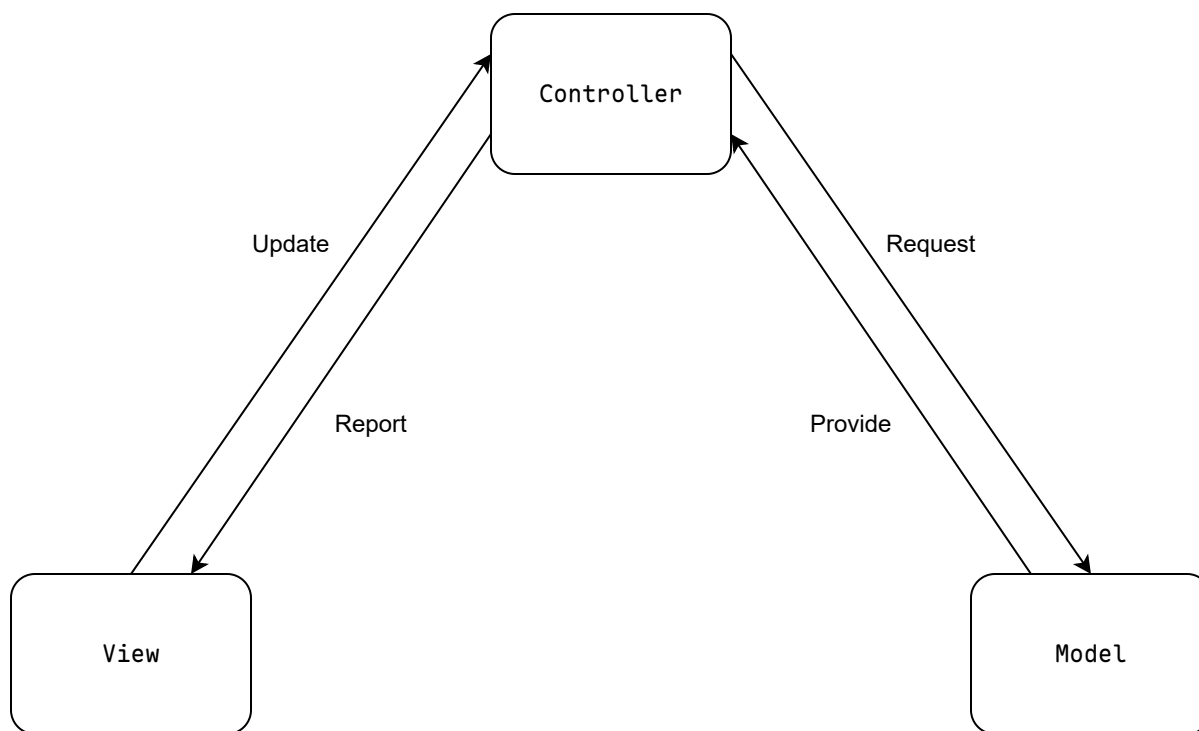


ERA-Engine (Godot) 将 FSM 模式植入系统，为 ERA 和 类ERA 游戏提供了全新的编程模式: **FSM 模式**。

在 ERA-Engine 中，任何状态信息 (如界面的布局 and 显示内容，角色的状态等) 被抽象为状态 **state**，状态间的转换则可称为转置 **transition**: UI 的变换 (如某个特定按钮的消失或出现、页面的切换等)，游戏进度的推进，诸如此类。因此，事实上，就如有限状态机所说的，在这一模式下，游戏本身的处理和推进本质上也可理解为不同状态 **state** 之间的切换。

MVC 模式

模型-视图-控制器 (Model-View-Controller, MVC) 是一种设计模式，用于将应用程序分为三个主要部分：模型、视图和控制器。



通俗而言，视图 **view** 是用户所见的内容，模型 **model** 是用户所操作的数据，控制器 **controller** 则是用户和视图、模型之间的桥梁，程序运行的逻辑。在视图 **view** 更新时，控制器 **controller** 会执行主要逻辑和处理，在模型 **model** 中查询和更新数据，并且将之返回给视图 **view**，从而使用户看到操作或变化的结果。

通过这样的流程，程序可以被拆分为三个模块，从而帮助作者厘清思路，令代码结构更模块化，代码和程序更加稳定可靠。

系统架构

概览

总得来说，在 ERA-Engine (Godot) 中创作游戏，你需要关注一下两个主要的部分：

- **状态 (State)**: 界面的布置和内容，角色物品等的状态
- **数据 (Data)**: 所有的数据文件，包括角色 **character**、口上 **kojo**、物品 **item** 等

随后，基于这二者，你可以自由组织不同状态下界面显示单元 **UI Unit** 的排布，以及基于有限状态机的计算、转换，进而构成整个游戏的运行逻辑。



TIP

想要快速开始？看看[入门教程](#)吧！

ERA-Engine (Godot) 的功能特性

基于 ERA-Engine 的设计，ERA-Engine (Godot) 提供了以下功能特性：

编写模式

ERA-Engine (Godot) 提供了两种编写模式：**ERA 模式**和**FSM 模式**。

ERA 模式

ERA 模式提供了类似于 ERA 游戏和类 ERA 游戏的传统编写方式。在编写时的体验会与 [Emuera](#) 游戏或 [ERAElectron](#) 游戏相似。

FSM 模式

FSM 模式，也就是有限状态机模式 (不是飞天意面 (Flying Spaghetti Monster, FSM)!)，是我们推荐的编写模式。在这一模式下，制作者们应该可以充分发挥 ERA-Engine 的效能，并且以更现代的方式编写和创作游戏。

设置状态

概述

在 **FSM 模式** 下，游戏本质上是状态 **state** 间的转移，因此显而易见的，制作游戏的主要工作便是各个状态的脚本。

创造你的第一个状态

1. 创建并编辑 **main_state.gd**

2. 编写状态

- 创建一个 **main_state** 函数，作为这个状态的入口：

```
func main_state():  
    TXT("era靛紫档案") # Create a text unit  
    BOX([ # Create a series of units, pack in a box  
        BTN("开始游戏", "start_game"), # Create a button unit, and set corresponding ne  
        BTN("读取存档", "load_game")  
    ])
```

你可以使用 **BOX** 来包裹一组界面显示单元，并且用这种方式调整它们的布局。对于各个UI元素，诸如 **TXT**、**BTN**、**BOX** 等，参考API文档来查看更进一步的参数选项。

- 可以注意到的是，一个按钮 **BTN** 显然会带来状态的转换，因此在它声明时也指定了它被点击时应当转向的新状态的名字。

3. 下一个状态函数 1.使用如下代码来设置下一个状态 (**start_game** 和 **load_game**):

```
func start_game():  
    TXT("你喜欢什么样的主角? ")  
    BTN("喜爱淑女的主角", "select_char_0")  
    BTN("财大气粗的主角", "select_char_1")  
    BTN("神父气质的主角", "select_char_2")  
    BTN("流氓热血的主角", "select_char_3")
```

```
func load_game():  
    TXT("读取存档")  
    BTN("读取不了，返回", "main_state")
```

```
# Continue with select_char_0, select_char_1, select_char_2, select_char_3...
```

现在，你的 **main_state** 的按钮应该可以正常工作了。如果你希望继续完成编码工作，那么 **start_game** 部分声明了更多状态，你可以继续完善这些代码。 2. 遵循这样的工作流程，你就可以逐步构建你的第一个游戏

数据文件

在 **FSM 模式** 下编写游戏的另一个重要部分是数据文件。无论是口上、角色数据、物品和装备数据，亦或是其他更多类型的信息，都需要存储为数据文件以供程序查询。

目前，ERA-Engine支持以下数据文件格式：

已经支持的格式

CSV

CSV（逗号分隔值）是一种文本数据格式，它使用逗号分隔不同的值，并使用换行符分隔记录。CSV 数据以纯文本形式存储表格数据（数字和文本），其中每一行通常代表一条数据记录。

CSV 格式最适合表示记录集或记录序列，其中每条记录都具有相同的字段列表。这对应于关系数据库中的单个关系，或者对应于典型电子表格中的数据（但不包括计算结果）。

CSV 格式是保存数据最简单的格式，也是 Emuera 游戏保存数据的方式。

计划支持的格式

INI

INI 文件是一种计算机软件的配置文件，它由纯文本构成，结构和语法以键值对的形式组织成不同的部分。

INI 文件易于阅读和解析，因此适用于不太复杂的配置文件。

Godot 原生支持 INI 格式的文件，因此我们很快也将支持这种数据文件管理方式。

XML

XML（Extensible Markup Language, 可扩展标记语言）是一种用于存储、传输和重构数据的标记语言和文件格式。它定义了一套规则，用于以既人可读又机器可读的格式对文档进行编码。

XML 已广泛应用于互联网数据交换。数百种使用 XML 语法的文档格式已被开发出来，包括 RSS、Atom、Office Open XML、OpenDocument、SVG、COLLADA 和 XHTML。

与 CSV 相比，XML 可以提供更易读、结构化的数据组织方式。

产品设定

关于 ERA-Engine (Godot) 的进阶信息

这一章节会提供更面向专业人士或需要进一步自定义的用户的，关于 ERA-Engine (Godot) 的信息。

设计原则

有限状态机 (FSM)

数据传输对象 (DTO)

数据传输对象 (Data Transfer Object, DTO)

系统架构

进阶功能